

Laporan Milestone 3
Tugas Besar
IF2224-Teori Bahasa Formal dan Otomata



Disusun oleh:

Muhammad Alfansya	13523005
Orvin Andika Ikhsan Abhista	13523017
Dzaky Aurelia Fawwaz	13523065
Ferdin Arsenarendra Purtadi	13523117

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025

1. Landasan Teori

Bahasa pemrograman adalah bahasa yang selalu digunakan untuk melakukan operasi-operasi yang melibatkan komputasi. Terkait bahasa pemrograman ini, proses untuk mengeksekusi bahasa pemrograman tersebut tidaklah mudah. Terdapat banyak proses yang perlu dilakukan oleh *compiler* supaya kode yang ditulis benar dan bisa memberikan output yang sesuai dengan bahasa pemrograman yang ditulis.

1.1. Analisis Semantik

Analisis semantik adalah tahap ketiga dalam proses kompilasi setelah analisis sintaks. Pada tahap ini, compiler memeriksa struktur pohon Abstract Syntax Tree untuk memastikan bahwa program tersebut memiliki makna yang konsisten dan mematuhi aturan logika bahasa pemrograman. Berbeda dengan analisis sintaks yang hanya mengecek tata bahasa, analisis ini berfokus pada validitas konteks, seperti pemeriksaan tipe dan aturan lingkup.

Tujuan utama dari analisis semantik adalah untuk memverifikasi bahwa struktur gramatikal yang benar juga masuk akal secara operasional. Beberapa hal yang dilakukan oleh analisis ini antara lain adalah memastikan variabel telah dideklarasikan sebelum digunakan, fungsi dipanggil dengan jumlah dan tipe parameter yang sesuai, dan operasi antar tipe data dilakukan secara kompatibel.

Apabila terdapat ketidakkonsistenan logika atau pelanggaran aturan tipe, *analyzer* akan mengirim pesan *semantic error*. Tahap ini bertujuan untuk menjamin keamanan dan kebenaran program sebelum diterjemahkan ke kode antara pada tahap berikutnya.

1.2. Abstract Syntax Tree

Abstract Syntax Tree merupakan representasi hierarkis dari struktur sintaksis program yang telah disederhanakan dari parse tree. Berbeda dengan parse tree yang memuat setiap detail langkah derivasi tata bahasa termasuk elemen sintaksis pendukung seperti tanda kurung, titik koma, atau kata kunci pemisah yang sudah tidak lagi relevan secara logika, AST bersifat abstrak karena hanya mempertahankan elemen-elemen yang memiliki makna operasional dan semantik bagi program. Dalam struktur pohon ini, simpul-simpul internal merepresentasikan operator operasi atau konstruksi aliran kendali seperti pernyataan kondisional dan perulangan, sedangkan simpul daun merepresentasikan operand dasar seperti nama variabel atau nilai konstanta. Struktur yang padat ini memungkinkan kompilator memahami logika program secara langsung tanpa terganggu oleh "derau" sintaksis.

Peran AST sangat krusial sebagai jembatan antara tahap analisis sintaks dengan tahap-tahap selanjutnya seperti analisis semantik, optimasi, dan pembangkitan kode. Secara implementasi, AST sering kali dibangun menggunakan pola desain komposit di mana setiap jenis simpul memiliki struktur data spesifik sesuai fungsinya, namun tetap berbagi antarmuka umum agar dapat diproses secara seragam. Hal ini memfasilitasi penerapan algoritma traversal seperti pola visitor, di mana penganalisa semantik akan

berjalan menelusuri setiap cabang pohon untuk memvalidasi tipe data dan logika, menjadikan AST sebagai struktur data utama yang menjamin kode program tidak hanya benar secara tata bahasa tetapi juga valid secara makna sebelum diterjemahkan ke bahasa mesin.

1.3. Tabel Simbol

Symbol table adalah struktur data fundamental dalam compiler yang digunakan untuk menyimpan informasi tentang identifier yang dideklarasikan dalam program. Struktur ini berfungsi sebagai database yang menyimpan atribut-atribut penting dari setiap identifier seperti nama, tipe data, scope level, dan informasi lainnya yang diperlukan untuk semantic analysis dan code generation. Dalam konteks Pascal-S compiler, symbol table tidak hanya menyimpan informasi identifier tetapi juga mengatur hubungan antar scope dan memfasilitasi proses type checking.

Implementasi symbol table dalam Pascal-S menggunakan tiga tabel terpisah yang saling berkaitan untuk mengakomodasi berbagai jenis informasi:

- a. tab (Identifier Table) - Menyimpan informasi tentang semua identifier dalam program termasuk konstanta, variabel, prosedur, dan fungsi. Setiap entry memiliki atribut:
 - name: Nama identifier
 - obj: Jenis objek (constant, variable, type, procedure, function, program)
 - type: Tipe data dari identifier (integer, real, boolean, char, string, array, record)
 - ref: Pointer/index ke tabel lain (atab untuk array, btab untuk procedure/record)
 - nrm: Menandai variabel normal (=1) atau parameter by-reference (=0)
 - lev: Lexical level tempat identifier dideklarasikan (0=global, 1=level pertama, dst)
 - adr: Address/offset untuk alokasi memory atau nilai konstanta
 - link: Index ke identifier sebelumnya dalam scope yang sama (membentuk linked list)
- b. btab (Block Table) - Menyimpan informasi tentang setiap block dalam program (prosedur, fungsi, record). Atribut meliputi:
 - last: Pointer ke identifier terakhir yang dideklarasikan dalam block ini
 - lpar: Pointer ke parameter terakhir dari prosedur/fungsi
 - psze: Total ukuran parameter dalam byte/unit memori
 - vsze: Total ukuran variabel lokal dalam byte/unit memori
- c. atab (Array Table) - Menyimpan informasi khusus tentang tipe array, mencakup:
 - xtyp/index_type: Tipe data index array (biasanya integer)
 - etyp/element_type: Tipe data elemen array
 - eref: Pointer ke detail tipe elemen jika elemen adalah tipe komposit
 - low: Batas bawah index array

- high: Batas atas index array
- elsz/element_size: Ukuran satu elemen array
- size: Total ukuran array

Symbol table mengimplementasikan lexical scoping menggunakan display stack yang menyimpan block index untuk setiap level scope (display[level] → block index di btab). Method enter_block() meng-increment level, membuat entry baru di btab (last=0, lpar=0, psze=0, vsze=0), dan append block index ke display. Sebaliknya, leave_block() men-decrement level dan pop display stack, membuat identifier dalam block tidak lagi accessible.

Identifier dalam satu block dihubungkan dengan linked list menggunakan field link yang menunjuk ke identifier sebelumnya, dengan btab[block_idx]["last"] sebagai head. Struktur ini memungkinkan iterasi efisien dan duplicate checking dalam scope yang sama.

Symbol table diinisialisasi dengan reserved words dan built-in procedures pada index 0-30, meliputi reserved types (integer, real, boolean, char, string) sebagai ObjType.TYPE, keywords bahasa (program, variabel, mulai, selesai, dll) sebagai marker, dan built-in procedures (writeln, readln, write, read) sebagai ObjType.PROCEDURE. User-defined identifiers dimulai dari index 31 (user_id_start = 31).

1.4. Type dan Scope Checking

Semantic analysis melakukan dua pemeriksaan utama, yaitu type checking untuk memastikan konsistensi tipe data, dan scope checking untuk memastikan identifier digunakan dalam scope yang tepat. Kedua proses ini dilakukan secara bersamaan saat traversal AST menggunakan visitor pattern, dimana setiap node dikunjungi dan divalidasi sesuai aturan semantik Pascal-S.

1.4.1. Type Checking

Type checking adalah proses verifikasi bahwa operasi-operasi dalam program menggunakan tipe data yang kompatibel sesuai aturan bahasa. Proses dilakukan dengan mengunjungi setiap node dalam AST dan menghitung tipe data hasil dari ekspresi berdasarkan tipe operand-nya. Sistem tipe Pascal-S mencakup:

- Tipe Dasar: INTEGER, REAL, BOOLEAN, CHAR, STRING
- Tipe Komposit: ARRAY, RECORD
- Tipe Khusus: VOID untuk prosedur dan error recovery, RANGE untuk subrange types

1.4.2. Aturan Type Checking

- Operasi Aritmatika (+, -, *, /, bagi, mod)
Menggunakan type promotion dimana jika salah satu operand bertipe REAL maka hasil dipromote menjadi REAL. Jika kedua operand INTEGER maka hasil tetap INTEGER. Method arithmetic_type_promotion() mengimplementasikan logika ini.

- Operasi Relasional (=, <>, <, <=, >, >=)
Operand harus kompatibel (sama atau dapat dipromote), hasil selalu bertipe BOOLEAN.
- Operasi Logical (dan, atau, tidak)
Operand harus bertipe BOOLEAN, hasil juga BOOLEAN.
- Assignment Statement
Tipe target dan tipe value harus kompatibel. Method `is_type_compatible()` memverifikasi: tipe sama → OK, target REAL dan source INTEGER → OK (implicit promotion), salah satu VOID → OK (untuk error recovery), selain itu → incompatible.

1.4.3. Scope Checking

Scope checking memastikan setiap identifier yang digunakan telah dideklarasikan dan accessible dari titik penggunaan. Method `find_identifier(name)` melakukan lookup dengan aturan lexical scoping. Dimulai dari level saat ini, traverse linked list identifier menggunakan field link. Jika tidak ditemukan, turun ke level yang lebih rendah (outer scope). Loop berlanjut hingga level 0 (global scope). Jika masih tidak ditemukan, cek reserved words di index 0-30. Return None jika identifier benar-benar tidak ada (undefined variable error).

Implementasi ini secara otomatis mengikuti prinsip shadowing dimana inner scope dapat mendefinisikan ulang identifier yang ada di outer scope. Lookup berhenti segera setelah menemukan match pertama, sehingga identifier di inner scope "menutupi" identifier dengan nama sama di outer scope.

1.4.4. Decorated AST

Hasil dari type & scope checking adalah Decorated AST dimana setiap node minimal memiliki informasi:

- `data_type`: Tipe data hasil evaluasi node (dari enum `BaseType`)
- `tab_index`: Index entry di symbol table jika node mereferensikan identifier
- `block_index`: Index block di btab untuk informasi scope

Decorated AST ini kemudian dapat digunakan oleh fase berikutnya (code generation) karena sudah mengandung semua informasi tipe dan referensi yang diperlukan untuk menghasilkan kode mesin.

1.5. Visit Function

Proses inti dari Semantic Analysis yaitu pembentukan AST, pengaksesan Symbol Table, dan pengecekan tipe/scope, dilakukan melalui mekanisme Semantic Visitor. Visitor ini merupakan implementasi dari pola desain Visitor yang menelusuri (traversal) Parse Tree yang dihasilkan oleh parser secara top-down atau Depth First. Setiap jenis

non-terminal node dalam Parse Tree atau AST akan memiliki fungsi penelusuran (traversal) yang terkait, yang disebut fungsi `visit_<node>`.

Fungsi `visit_<node>` bertindak sebagai Semantic Action yang dieksekusi saat node tersebut dikunjungi, dengan menjalankan langkah-langkah berikut secara berurutan:

1. Memanggil `visit` pada child-node: Menerapkan rekursi untuk menelusuri sub-pohon di bawah node saat ini.
2. Mengakses atau memperbarui Symbol Table: Melakukan operasi penting seperti push scope baru ke stack Symbol Table saat memasuki blok kode baru, memasukkan variabel/fungsi ke Symbol Table saat deklarasi, atau melakukan lookup pada identifier saat ditemukan.
3. Menghitung atribut/tipe semantik: Mengkalkulasi tipe data dari ekspresi yang dikunjungi dan memastikan konsistensi tipe data (type compatibility).
4. Menambahkan anotasi pada node: Mendekorasi AST (Decorated AST) dengan informasi semantik yang telah divalidasi, termasuk tipe ekspresi dan referensi ke Symbol Table.

2. Perancangan & Implementasi

2.1. Implementasi

2.1.1. Struktur Symbol Table

Symbol Table diimplementasikan melalui kelas `SymbolTable` yang menampung tiga tabel utama (`tab`, `btab`, dan `atab`) dan mekanisme manajemen scope (level dan `display`).

2.1.1.1. Symbol Table Container

```
class SymbolTable:
    def __init__(self):
        self.tab: List[Optional[Dict[str, Any]]] =
        []

        self.btab: List[Dict[str, Any]] = []
        self.atab: List[Dict[str, Any]] = []

        self._init_reserved_words()

        self.display: List[int] = []
        self.level: int = -1 # Lexical level
        # ... (other initializations)
```

- `self.tab`
Identifier Table. Menyimpan entri untuk semua identifier yang dideklarasikan, termasuk tipe dasar dan reserved words.
- `self.btab`

Block Table. Menyimpan informasi tentang setiap blok kode (program utama, prosedur, fungsi, record)

- self.atab
Array Table. Menyimpan detail tentang tipe array (batasan, tipe indeks, tipe elemen)
- self.level
Lexical Level saat ini. Digunakan untuk melacak kedalaman scope.
- self.display
Stack yang menyimpan indeks btab dari blok yang saat ini aktif pada setiap lexical level

2.1.1.2. Struktur Entry

a. Identifier table (tab)

Setiap entri identifier yang baru dimasukkan memiliki struktur berikut (dikelola oleh enter_identifier):

```
# Contoh entri dalam self.tab
{
    "name": name,           # Nama
    identifier
    "obj": obj_type,        # Kelas objek
    (misalnya ObjType.VARIABLE)
    "type": data_type,      # Tipe dasar
    (misalnya BaseType.INTEGER)
    "ref": ref,             #
    Pointer/indeks ke btab atau atab jika
    tipe komposit
    "nrm": nrm,             # Normal (1)
    atau by-reference (0)
    "lev": self.level,      # Tingkat
    leksikal
    "adr": adr,             # Offset
    variabel di stack frame [cite: 1096,
    1101]
    "link": link_value      # Indeks
    identifier sebelumnya dalam scope yang
    sama
}
```

b. Block Table (btab)

Setiap kali scope baru dibuka, entri btab baru dibuat (oleh enter_block):

```
# Contoh entri dalam self.btab
{
```

```

        "last": 0,          # Indeks identifier
        terakhir yang dideklarasikan di blok ini
        [cite: 1103]
        "lpar": 0,          # Indeks parameter
        terakhir (untuk prosedur/fungsi) [cite:
        1103]
        "psze": 0,          # Total ukuran
        parameter [cite: 1103]
        "vsze": 0           # Total ukuran
        variabel lokal [cite: 1109]
    }

```

2.1.1.3. Manajemen Scope

Manajemen scope leksikal diatur oleh `enter_block` dan `leave_block`, yang memodifikasi `self.level` dan `self.display`.

a. `enter_block`

Method `enter_block()` digunakan untuk masuk ke scope baru dalam program. Method ini meng-increment level counter, membuat entry baru di `btab` dengan semua field bernilai nol (`last=0`, `lpar=0`, `psze=0`, `vsze=0`), lalu meng-append block index ke `display` stack untuk lookup cepat. Method mengembalikan block index yang disimpan di AST nodes. Biasanya dipanggil saat memproses program utama, procedure, function, atau compound statement yang membuat scope baru.

```

def enter_block(self) -> int:
    self.level += 1
    block_index = len(self.btab)

    # 1. Tambah entry baru ke btab
    self.btab.append({
        "last": 0, "lpar": 0, "psze":
0, "vsze": 0
    })

    # 2. Tambah indeks btab ke stack
display
    self.display.append(block_index)
    return block_index

```

b. `leave_block`

Method `leave_block()` adalah kebalikan dari `enter_block()`, digunakan untuk keluar dari scope saat ini. Method ini men-decrement level counter untuk kembali ke parent scope, lalu melakukan pop pada `display` stack. Cleanup identifiers di block

ini terjadi secara implicit karena lookup hanya traverse level yang masih aktif. Dipanggil setelah selesai memproses body procedure, function, atau block.

```
def leave_block(self):
    if self.level > 0:
        self.level -= 1
        self.display.pop()
```

c. enter_identifier

Method enter_identifier() memasukkan identifier baru ke symbol table. Prosesnya dimulai dengan mengambil tab_index dari next_user_id lalu increment counter. Untuk VARIABLE, method meng-assign address dari next_adr dan increment sesuai size. Method mengambil current block dari display stack dan nilai last sebagai head linked list, lalu expand tab jika perlu dengan placeholder None. Entry baru dibuat di tab[tab_index] dengan semua field (name, obj_type, data_type, ref, nrm, level, address, link). Field link diisi dengan previous last untuk membentuk linked list identifier dalam block. Method kemudian update current_block["last"] menjadi tab_index sebagai head baru. Untuk CONSTANT, nilai disimpan di const_values dict, sedangkan untuk VARIABLE, vsze di-increment. Method mengembalikan tab_index untuk disimpan di AST node.

```
def enter_identifier(self, name: str,
                    obj_type: ObjType, data_type: int,
                                ref: int = 0,
                    nrm: int = 1, size: int = 1,
                                const_value: Any
                    = None) -> int:
    tab_index = self.next_user_id
    self.next_user_id += 1

    if obj_type == ObjType.VARIABLE:
        adr = self.next_adr
        self.next_adr += size
    else:
        adr = 0

    current_block_idx =
self.display[self.level]
    current_block =
self.btab[current_block_idx]
    prev_last = current_block["last"]
```

```

        if tab_index >= len(self.tab):
            while len(self.tab) <=
tab_index:
                self.tab.append(None)

        link_value = prev_last

        self.tab[tab_index] = {
            "name": name,
            "obj": obj_type,
            "type": data_type,
            "ref": ref,
            "nrm": nrm,
            "lev": self.level,
            "adr": adr,
            "link": link_value
        }
        if obj_type == ObjType.CONSTANT
and const_value is not None:
            self.const_values[name] =
const_value
            current_block["last"] = tab_index

        if obj_type == ObjType.VARIABLE:
            current_block["vsze"] += size

        return tab_index

```

d. `find_identifier`

Method `find_identifier()` mencari identifier berdasarkan nama dengan aturan scoping. Pencarian dilakukan dengan loop dari level saat ini turun ke level 0, mengimplementasikan prinsip inner scope diutamakan. Untuk setiap level, method mengambil `block_index` dari `display` dan `last` sebagai starting point, lalu traverse linked list menggunakan field `link`. Jika match ditemukan, index dikembalikan. Jika tidak ketemu di user identifiers, method mencari di reserved words (index 0-30). Jika tetap tidak ketemu, method mengembalikan `None`. Implementasi ini otomatis mengikuti lexical scoping dimana inner scope men-shadow outer scope.

```

def find_identifier(self, name: str) ->
Optional[int]:
    for level in range(self.level,
-1, -1):
        block_index =

```

```

self.display[level]
    last_idx =
self.btab[block_index]["last"]

        current_idx = last_idx
        while current_idx >=
self.user_id_start:
    entry =
self.tab[current_idx]
    if entry is not None and
entry["name"] == name:
        return current_idx
    if entry is None:
        break
    current_idx =
entry["link"]

        for i in
range(self.user_id_start):
            if i < len(self.tab) and
self.tab[i] and self.tab[i]["name"] ==
name:
                return i

        return None

```

e. `get_constant_value`

Method `get_constant_value()` adalah helper untuk retrieve nilai aktual konstanta. Method melakukan lookup di dictionary `const_values` dengan nama sebagai key, mengembalikan nilai jika ditemukan atau `None` jika tidak ada. Digunakan saat semantic analyzer perlu me-resolve referensi konstanta dalam expressions.

```

def get_constant_value(self, name: str)
-> Optional[Any]:
    return
self.const_values.get(name)

```

f. `enter_array`

Method `enter_array()` membuat entry di array table saat compiler menemukan deklarasi array. Method menghitung total size dengan formula $(high - low + 1) * element_size$, mengambil `atab_index` dari panjang current atab, lalu meng-append dictionary baru berisi `index_type`, `element_type`, `eref`, `low`, `high`, `element_size`, dan `size`. Method mengembalikan `atab_index` yang disimpan di field `ref identifier array`.

Saat variabel array didefinisikan, dua entry dibuat: entry di tab dengan type=ARRAY.value dan ref=atab_index, serta entry di atab dengan detail lengkap struktur array. Pemisahan ini memungkinkan representasi efisien dimana informasi umum identifier ada di tab, sementara informasi spesifik array ada di atab.

```
def enter_array(self, index_type: int,
                element_type: int,
                low_bound: int,
                high_bound: int,
                element_size: int = 1)
-> int:
    array_size = (high_bound -
low_bound + 1) * element_size

    atab_index = len(self.atab)
    self.atab.append({
        "index_type": index_type,
        "element_type": element_type,
        "eref": 0,
        "low": low_bound,
        "high": high_bound,
        "element_size": element_size,
        "size": array_size
    })

    return atab_index
```

2.1.2. Semantic Analyzer Class

Kelas ini menginisialisasi Symbol Table dan error list. Metode analyze adalah titik masuk utama proses analisis semantik.

```
class SemanticAnalyzer:
    def __init__(self):
        self.symbol_table = SymbolTable()
        self.current_ast: Optional[ASTNode] = None
        self.errors: List[str] = []

    def analyze(self, parse_tree: ParseNode) ->
ASTNode:
        self.errors.clear()

        global_block_idx =
self.symbol_table.enter_block() # Buka scope global
        self.current_ast = self.visit(parse_tree)
```

```
        self.symbol_table.leave_block() # Tutup
scope global

        return self.current_ast
```

- Initialization
SemanticAnalyzer membuat satu instance SymbolTable untuk digunakan sepanjang proses.
- Scope Management
Metode analyze secara eksplisit memanggil self.symbol_table.enter_block() di awal dan self.symbol_table.leave_block() di akhir, memastikan program utama berjalan dalam scope global.
- Traversal Start
Proses dimulai dengan memanggil self.visit(parse_tree) pada root node Parse Tree.

2.1.2.1. Mekanisme Visitor

Pola Visitor diimplementasikan menggunakan dynamic dispatch (getattr) untuk mengarahkan setiap ParseNode ke metode visit_ yang sesuai.

```
def visit(self, node: ParseNode) -> ASTNode:
    clean_node_name =
self.clean_name(node.name).replace("-", "_")

    # ... (logic untuk menangani nama node
yang kompleks)

    method_name =
f'visit_{clean_node_name}'
    method = getattr(self, method_name,
self.visit_default)

    try:
        return method(node)
    # ... (error handling)

    def visit_default(self, node: ParseNode)
-> ASTNode:
        """Default visitor - just traverse
children"""
        ast_node = ASTNode(node.name)
        for child in node.children:
            res = self.visit(child)
            if res:
```

```
        ast_node.add_child(res)
    return ast_node
```

visit(node) mengambil nama ParseNode (misalnya, <assignment-statement>), mengubahnya menjadi nama metode (visit_assignment_statement), dan memanggilnya dan visit_default akan membuat node AST umum dan melanjutkan traversal rekursif ke semua anak jika tidak ada metode khusus yang ditemukan (misalnya, untuk non-terminal yang hanya bersifat struktural).

2.1.2.2. Program dan Structure Visitor

- visit_program(node)
Memproses root program node. Extract nama program dari header, masukkan program ke symbol table sebagai ObjType.PROGRAM, buat main block untuk program, proses declaration-part dan compound-statement sebagai children, lalu leave block
- visit_declaration_part(node)
Mengumpulkan semua deklarasi dalam program. Memproses const-declaration, type-declaration, var-declaration, dan subprogram-declaration. Setiap hasil deklarasi ditambahkan sebagai child ke node "Declarations"
- visit_block(node)
Memproses block yang terdiri dari declaration-part dan compound-statement. Set block_index dari display stack current level, berguna untuk tracking scope level di code generation nantinya
- visit_compound_statement(node)
Memproses blok mulai-selesai. Ambil current block index dari display stack, visit statement-list di dalamnya, tambahkan semua statement sebagai children
- visit_statement_list(node)
Memproses daftar statement yang dipisahkan semicolon. Dispatch ke visitor spesifik untuk setiap jenis statement (assignment, if, while, for, dll). Skip node SEMICOLON karena hanya separator

2.1.2.3. Declaration Visitor

- visit_var_declaration(node) - Memproses keyword 'variabel' dan semua var-item di bawahnya. Skip keyword node, proses setiap var-item, extract VarDeclNode dari hasil, set block_index sesuai level saat ini

- `visit_var_item(node)` - Memproses satu item deklarasi variabel (contoh: `x, y: integer`). Extract identifier list dan type, untuk setiap identifier: cek duplikasi di level yang sama, masukkan ke symbol table dengan `enter_identifier`, buat `VarDeclNode` dengan `tab_index` dan `data_type` yang tepat
- `visit_type(node)` - Resolve tipe data dari node. Untuk built-in types (integer, real, boolean, char, string), mapping dari token value ke `BaseType` enum. Untuk array-type, dispatch ke `visit_array_type`. Return `ASTNode` dengan `data_type` yang sesuai
- `visit_array_type(node)` - Memproses deklarasi tipe array. Parse index-specification untuk mendapat low dan high bound, visit element type, validasi bounds (`low <= high`), buat entry di atab dengan `enter_array`, return `ArrayType` node dengan `array_ref` ke atab
- `visit_const_declaration(node)` - Memproses deklarasi konstanta. Iterate semua const-item children, visit masing-masing, tambahkan hasil ke collection
- `visit_const_item(node)` - Memproses item konstanta individual (contoh: `MAX = 100`). Extract identifier dan const-value, handle referensi ke konstanta lain (resolve nilainya), cek duplikasi identifier, masukkan ke symbol table dengan `const_value`, simpan nilai di `const_values` dict untuk lookup nanti
- `visit_const_value(node)` - Extract nilai literal konstanta. Handle `NUMBER` token (int atau real berdasarkan ada tidaknya titik desimal), handle `STRING_LITERAL` (deteksi char vs string berdasarkan panjang), set `data_type` dan value yang tepat

2.1.2.4. Variables dan Expression Visitor

- `visit_variable(node)`
Memproses referensi variabel. Jika ada `LBRACKET` berarti array element: extract array name, find di symbol table, ambil element type dari atab, parse index expression(s), validasi bounds checking. Jika variabel biasa: find di symbol table, buat `VariableNode` dengan `data_type` dari symbol table. Error jika undefined
- `visit_assignment_statement(node)`
Memproses statement assignment (`:=`). Parse target identifier (harus ada di symbol table), visit expression di rhs, type checking antara target dan value (compatible types), buat `AssignmentNode` dengan target dan value sebagai children
- `visit_expression(node)`
Memproses ekspresi level tertinggi. Jika hanya 1 child berarti simple-expression, langsung visit itu. Jika 3+ children berarti ada

relational operator ($>$, $<$, $=$, dll): visit left simple-expression, ambil operator, visit right simple-expression, buat BinaryExpressionNode dengan data_type BOOLEAN

- visit_simple_expression(node)
Memproses chain of or-terms dengan operator 'atau'. Filter children kosong, visit first or-term sebagai result awal, lalu loop: untuk setiap operator 'atau', visit or-term berikutnya, buat BinaryExpressionNode untuk 'atau' dengan left=result dan right=or-term baru, update result. Left-to-right evaluation
- visit_or_term(node)
Memproses chain of terms dengan operator additive (+, -). Filter children kosong, visit first term, lalu loop setiap operator: visit term berikutnya, arithmetic type promotion ($\text{int} + \text{real} \rightarrow \text{real}$), buat BinaryExpressionNode dengan operator +/-, update result
- visit_term(node)
Memproses chain of and-factors dengan operator 'dan'. Filter children kosong, visit first and-factor, loop setiap 'dan': visit and-factor berikutnya, buat BinaryExpressionNode untuk 'dan' dengan data_type BOOLEAN, update result
- visit_and_factor(node)
Memproses chain of factors dengan operator multiplicative (*, /, bagi, mod). Filter children kosong, visit first factor, loop setiap operator: visit factor berikutnya, type promotion, buat BinaryExpressionNode, update result
- visit_factor(node)
Memproses unit terkecil ekspresi. Handle NUMBER literal (int atau real), CHAR_LITERAL/STRING_LITERAL (char), IDENTIFIER (lookup di symbol table, error jika undefined), parentheses (recursive visit expression di dalam), unary 'tidak' (buat UnaryExpressionNode dengan operator "tidak")

2.1.2.5. Control Flow Visitor

- visit_if_statement(node)
Memproses if-then-else statement. Visit condition expression (harus boolean, error jika tidak), visit then statement/compound-statement, visit else statement jika ada, buat IfStatement node dengan condition, then, else sebagai children (else optional)
- visit_while_statement(node)
Memproses while-do loop. Visit condition expression (type check: harus boolean), visit body statement/compound-statement, buat WhileStatement node dengan condition dan body sebagai children

- `visit_for_statement(node)`
Memproses for loop (untuk...ke atau turun_ke). Extract control variable (lookup di symbol table), visit start expression, detect 'turun_ke' untuk downto, visit end expression, type check control variable (harus integer atau char), visit body, buat ForStatementNode dengan `is_downto` flag dan 4 children: control var, start, end, body
- `visit_repeat_statement(node)`
Memproses repeat-until loop. Visit statement-list untuk body (bisa multiple statements), visit condition expression (harus boolean), buat RepeatStatement dengan body statements + condition sebagai children. Berbeda dari while karena condition di akhir

2.1.2.6. SubProgram Visitor

- `visit_subprogram_declaration(node)`
Wrapper sederhana yang meneruskan ke child pertama (bisa procedure atau function declaration)
- `visit_procedure_declaration(node)`
Memproses deklarasi prosedur. Extract nama prosedur, check duplicate, masukkan ke symbol table sebagai `ObjType.PROCEDURE` dengan type VOID, enter block baru untuk procedure scope, visit formal-parameter-list, visit block (body), simpan `block_index` ke tab entry, leave block setelah selesai
- `visit_function_declaration(node)`
Memproses deklarasi fungsi (mirip procedure tapi ada return type). Extract nama fungsi dan return type dari type node, check duplicate, masukkan ke symbol table sebagai `ObjType.FUNCTION` dengan return type yang sesuai, enter block baru, visit parameters dan body, leave block
- `visit_procedure_call(node)`
Memproses pemanggilan prosedur atau built-in (`writeln`, `readln`, `write`, `read`). Extract procedure name dari IDENTIFIER atau KEYWORD token, lookup di symbol table untuk dapat `tab_index`, visit parameter-list untuk ambil arguments, buat ProcedureCallNode dengan `procedure_name` dan parameters sebagai children
- `visit_function_call(node)`
Memproses pemanggilan fungsi. Extract function name, lookup di symbol table, ambil return type dari tab entry (untuk set `data_type` di AST), visit parameter-list, buat FunctionCall node

dengan return type yang tepat (penting untuk type checking expressions)

- `visit_parameter_list(node)`
Mengumpulkan parameter actual dalam procedure/function call.
Visit semua expression children, tambahkan ke collection.
Return ParameterList node yang children-nya adalah expression nodes

2.1.3. Abstract Syntax Tree (AST Nodes)

Kelas-kelas AST Nodes, yang didefinisikan dalam `ast_nodes.py`, berfungsi sebagai abstraksi hirarkis dari program sumber, menghilangkan detail sintaksis yang tidak perlu (e.g., semicolons, parentheses) sambil menyimpan informasi semantik yang vital. Semua node mewarisi dari kelas dasar `ASTNode` dan didekorasi (decorated) dengan atribut dari Semantic Analyzer.

`ASTNode` mendefinisikan atribut dasar yang harus dimiliki setiap node untuk menyimpan anotasi semantik:

```
@dataclass
class ASTNode:
    node_type: str
    children: List[ASTNode] = field(default_factory=list)
    token: Optional[Token] = None
    data_type: Optional['BaseType'] = None
    tab_index: int = -1
    block_index: int = -1
    # ...
```

2.1.3.1. Program Structure Node

- `ProgramNode` merepresentasikan root program dan extends dari `ASTNode`. Node ini memiliki field tambahan `name` untuk menyimpan nama program. Constructor menerima `node_type` dan `name`, lalu memanggil parent constructor. Method `__repr__()` mengembalikan format `"ProgramNode(name: 'nama_program')"` untuk identifikasi mudah saat debugging atau printing AST.
- `CompoundStatementNode` merepresentasikan blok mulai-selesai yang berisi statement-statement. Node ini tidak memiliki field tambahan khusus, hanya mewarisi dari `ASTNode`. Method `__repr__()` menampilkan `"CompoundStatement → block_index:N"` jika `block_index` sudah di-set, atau hanya `"CompoundStatement"` jika belum. Block index penting untuk code generation karena menunjukkan scope level.

2.1.3.2. Declaration Node

VarDeclNode merepresentasikan deklarasi variabel dan extends dari ASTNode. Node ini memiliki field identifier untuk menyimpan nama variabel. Constructor menerima node_type dan identifier, memanggil parent constructor untuk inisialisasi base fields. Method `__repr__()` mengembalikan format "VarDecl('nama_variabel')" yang ringkas untuk debugging.

2.1.3.3. Statement Nodes

- AssignmentNode merepresentasikan statement assignment (operator `:=`). Node ini tidak memiliki field tambahan, tapi selalu memiliki dua children: target (variabel yang di-assign) dan value (ekspresi nilai). Method `__repr__()` cukup kompleks karena memformat representasi dengan membaca children. Jika ada minimal 2 children, method akan format target dan value dengan logic khusus: target ditampilkan dengan quote jika VariableNode, value ditampilkan dengan format khusus jika BinaryExpressionNode (menampilkan operasi lengkap), hasil akhir format "Assign(target := value)". Jika children tidak lengkap, return "Assign(?)".
- IfStatementNode merepresentasikan if-then-else statement. Node tidak memiliki field tambahan, children-nya berisi condition (index 0), then-statement (index 1), dan optional else-statement (index 2). Method `__repr__()` menampilkan kondisi dan mengindikasikan ada tidaknya then/else part dengan format "IfStatement(condition then ... else ...)" atau hanya "IfStatement(?)" jika tidak ada children.
- WhileStatementNode merepresentasikan while-do loop. Node tidak memiliki field tambahan, children berisi condition (index 0) dan body statement (index 1). Method `__repr__()` menampilkan kondisi dengan format "WhileStatement(condition)" untuk identifikasi cepat jenis loop dan kondisinya.
- ForStatementNode merepresentasikan for loop dengan field tambahan `is_downto` (boolean) untuk membedakan loop "ke" atau "turun_ke". Children berisi control variable (index 0), start expression (index 1), end expression (index 2), dan body statement (index 3). Method `__repr__()` menampilkan semua informasi penting dengan format "ForStatement(var := start to/downto end)" yang sangat deskriptif untuk debugging.

2.1.3.4. Expression Node

- BinaryExpressionNode merepresentasikan operasi binary (dua operand) dan extends dari ASTNode. Node ini memiliki field operator untuk menyimpan operator (+, -, *, /, dan, atau, dll). Children selalu dua: left operand (index 0) dan right operand (index 1). Method `__repr__()` sangat detail karena memformat expression dengan operator mapping untuk pretty print (GT→'>', AND→' dan ', dll). Method juga memformat operand dengan logic khusus: VariableNode dengan quote, NumberNode/CharNode dengan repr mereka, nested expression dengan parentheses. Hasil format seperti "left+right" atau "left dan right" tergantung operator, dengan spacing yang sesuai untuk readability.
- UnaryExpressionNode merepresentasikan operasi unary (satu operand) dengan field operator untuk menyimpan operator (biasanya "tidak" untuk logical NOT). Children hanya satu yaitu operand. Method `__repr__()` menampilkan format "tidak (operand)" atau "operator (operand)" tergantung operator yang digunakan, dengan special handling untuk "tidak" agar lebih natural dalam Bahasa Indonesia.

2.1.3.5. Literal dan Variable Node

- VariableNode merepresentasikan referensi ke variabel dengan field identifier untuk nama variabel dan `is_array_element` (boolean) untuk membedakan variabel biasa dengan array element. Constructor menerima `node_type` dan identifier, memanggil parent constructor. Method `__repr__()` simpel mengembalikan `"Var('nama_variabel')"` untuk identifikasi cepat.
- NumberNode merepresentasikan literal angka (integer atau real) dengan field value (float) untuk menyimpan nilai numerik. Constructor menerima `node_type` dan value. Method `__repr__()` langsung return nilai sebagai string, sangat simpel karena literal tidak perlu informasi tambahan.
- CharNode merepresentasikan literal karakter dengan field value (string) untuk menyimpan satu karakter. Constructor menerima `node_type` dan value dengan default "Char". Method `__repr__()` mengembalikan `"'karakter'"` dengan single quote untuk menunjukkan ini char literal, konsisten dengan syntax Pascal.
- StringNode merepresentasikan literal string dengan field value untuk menyimpan teks string. Constructor menerima `node_type` dan value. Method `__repr__()` mengembalikan format `"String('teks')"` untuk membedakan dari char literal yang hanya ditampilkan dengan quote.

- BooleanNode merepresentasikan literal boolean dengan field value (bool) untuk menyimpan True/False dan identifier (string) untuk menyimpan representasi text ("true"/"false" atau "benar"/"salah"). Constructor menerima kedua parameter ini. Method `__repr__()` mengembalikan "Boolean('identifier')" yang menampilkan text representation, bukan nilai boolean langsung.

2.1.3.6. Procedure dan Function Node

ProcedureCallNode merepresentasikan pemanggilan prosedur atau built-in procedure (`writeln`, `readln`, dll) dengan field `procedure_name` untuk nama prosedur dan `is_user_defined` (boolean) untuk membedakan user-defined vs built-in. Constructor menerima `node_type` dan `procedure_name`. Children berisi parameter expressions yang dikirim saat pemanggilan. Method `__repr__()` simpel mengembalikan "ProcedureCall('nama_prosedur')" untuk identifikasi cepat jenis call.

3. Pengujian

3.1. Pengujian 1 : Program dengan Deklarasi Kompleks

Input
<pre> program Deklarasi; variabel i, j, k : integer; rataRata : real; isValid : boolean; hurufAwal : char; nilai : larik [1 .. 100] dari integer; matriks : larik [1 .. 100] dari real; mulai i := 1; hurufAwal := 'a'; isValid := tidak (i > 100); selesai. </pre>
Output

Input

```
===== DECORATED AST =====
ProgramNode(name: 'Deklarasi')
├── Declarations
│   ├── VarDecl('i') → tab_index:32, type:integer, lev:0
│   ├── VarDecl('j') → tab_index:33, type:integer, lev:0
│   ├── VarDecl('k') → tab_index:34, type:integer, lev:0
│   ├── VarDecl('rataRata') → tab_index:35, type:real, lev:0
│   ├── VarDecl('isValid') → tab_index:36, type:boolean, lev:0
│   ├── VarDecl('hurufAwal') → tab_index:37, type:char, lev:0
│   ├── VarDecl('nilai') → tab_index:38, type:array, lev:0
│   └── VarDecl('matriks') → tab_index:39, type:array, lev:0
├── Block → block_index:1, lev:1
│   ├── Assign('i' := (1) → type:integer) → type:void
│   │   ├── target 'i' → tab_index:32, type:integer
│   │   └── value (1) → type:integer
│   ├── Assign('hurufAwal' := ('a') → type:char) → type:void
│   │   ├── target 'hurufAwal' → tab_index:37, type:char
│   │   └── value ('a') → type:char
│   └── Assign('isValid' := (tidak 'i'>100) → type:boolean) → type:void
│       ├── target 'isValid' → tab_index:36, type:boolean
│       └── value (tidak 'i'>100) → type:boolean
```

Input

Identifier Table (tab):

Idx	Name	Obj	Type	Ref	Nrm	Lev	Adr	Link

0	integer	TYPE	INTEGER	0	1	0	0	0
1	real	TYPE	REAL	0	1	0	0	0
2	boolean	TYPE	BOOLEAN	0	1	0	0	0
3	char	TYPE	CHAR	0	1	0	0	0
4	string	TYPE	STRING	0	1	0	0	0
5	program	TYPE	VOID	0	1	0	0	4
6	variabel	TYPE	VOID	0	1	0	0	5
7	mulai	TYPE	VOID	0	1	0	0	6
8	selesai	TYPE	VOID	0	1	0	0	7
9	jika	TYPE	VOID	0	1	0	0	8
10	maka	TYPE	VOID	0	1	0	0	9
11	selain_itu	TYPE	VOID	0	1	0	0	10
12	selama	TYPE	VOID	0	1	0	0	11
13	lakukan	TYPE	VOID	0	1	0	0	12
14	untuk	TYPE	VOID	0	1	0	0	13
15	ke	TYPE	VOID	0	1	0	0	14
16	turun_ke	TYPE	VOID	0	1	0	0	15
17	larik	TYPE	VOID	0	1	0	0	16
18	dari	TYPE	VOID	0	1	0	0	17
19	prosedur	TYPE	VOID	0	1	0	0	18
20	fungsi	TYPE	VOID	0	1	0	0	19
21	konstanta	TYPE	VOID	0	1	0	0	20
22	tipe	TYPE	VOID	0	1	0	0	21
23	kasus	TYPE	VOID	0	1	0	0	22
24	rekaman	TYPE	VOID	0	1	0	0	23
25	ulangi	TYPE	VOID	0	1	0	0	24
26	sampai	TYPE	VOID	0	1	0	0	25
27	writeln	PROCEDURE	VOID	0	1	0	0	26
28	readln	PROCEDURE	VOID	0	1	0	0	27
29	write	PROCEDURE	VOID	0	1	0	0	28
30	read	PROCEDURE	VOID	0	1	0	0	29

Input								
31	Deklarasi	PROGRAM	VOID	0	1	0	0	0
32	i	VARIABLE	INTEGER	0	1	1	0	0
33	j	VARIABLE	INTEGER	0	1	1	1	32
34	k	VARIABLE	INTEGER	0	1	1	2	33
35	rataRata	VARIABLE	REAL	0	1	1	3	34
36	isValid	VARIABLE	BOOLEAN	0	1	1	4	35
37	hurufAwal	VARIABLE	CHAR	0	1	1	5	36
38	nilai	VARIABLE	ARRAY	0	1	1	6	37
39	matriks	VARIABLE	ARRAY	0	1	1	7	38

Block Table (btab):

Idx	Last	Lpar	Psize	Vsize
0	31	0	0	0
1	39	0	0	8

Array Table (atab):

Idx	IdxType	ElemType	Eref	Low	High	ElemSize	Size
0	INTEGER	INTEGER	0	1	10	1	10
1	INTEGER	REAL	0	1	10	1	10

Penjelasan
Analisis semantik berhasil melakukan binding variabel ke tabel simbol dan membuat decorated AST dengan informasi tipe data yang valid. Setiap node VarDecl dan Assign telah terhubung ke indeks tabel simbol yang tepat, serta dilakukan pemeriksaan kompatibilitas tipe pada operasi assignment untuk memastikan kesesuaian antara target dan nilai. Selain itu, manajemen memori divalidasi melalui Tabel Simbol yang telah merekam offset alamat (Adr) variabel secara berurutan dalam blok, dan Tabel Array (atab) sukses merekam struktur tipe data kompleks, menjamin konsistensi semantik sebelum masuk ke tahap pembangkitan kode.

3.2. Pengujian 2 : Program dengan Ekspresi yang Membutuhkan Urutan Operasi

Input
<pre>program Ekspresi; variabel a, b, c : integer; hasil : real; isTrue : boolean; mulai hasil := a + b * 2; hasil := (a + b) * 2; isTrue := (a > 0) atau (b < 0) dan (c = 10); selesai.</pre>
Output
<pre>===== DECORATED AST ===== ProgramNode(name: 'Ekspresi') ├── Declarations │ ├── VarDecl('a') → tab_index:32, type:integer, lev:0 │ ├── VarDecl('b') → tab_index:33, type:integer, lev:0 │ ├── VarDecl('c') → tab_index:34, type:integer, lev:0 │ ├── VarDecl('hasil') → tab_index:35, type:real, lev:0 │ └── VarDecl('isTrue') → tab_index:36, type:boolean, lev:0 └── Block → block_index:1, lev:1 ├── Assign('hasil' := ('a'+('b'*2)) → type:integer) → type:void │ ├── target 'hasil' → tab_index:35, type:real │ └── value ('a'+('b'*2)) → type:integer ├── Assign('hasil' := (('a'+'b')*2) → type:integer) → type:void │ ├── target 'hasil' → tab_index:35, type:real │ └── value (('a'+'b')*2) → type:integer └── Assign('isTrue' := ('a'>0atau('b'<0dan'c'=10)) → type:boolean) → type:void ├── target 'isTrue' → tab_index:36, type:boolean └── value ('a'>0atau('b'<0dan'c'=10)) → type:boolean</pre>

Input

Identififier Table (tab):

Idx	Name	Obj	Type	Ref	Nrm	Lev	Adr	Link

0	integer	TYPE	INTEGER	0	1	0	0	0
1	real	TYPE	REAL	0	1	0	0	0
2	boolean	TYPE	BOOLEAN	0	1	0	0	0
3	char	TYPE	CHAR	0	1	0	0	0
4	string	TYPE	STRING	0	1	0	0	0
5	program	TYPE	VOID	0	1	0	0	4
6	variabel	TYPE	VOID	0	1	0	0	5
7	mulai	TYPE	VOID	0	1	0	0	6
8	selesai	TYPE	VOID	0	1	0	0	7
9	jika	TYPE	VOID	0	1	0	0	8
10	maka	TYPE	VOID	0	1	0	0	9
11	selain_itu	TYPE	VOID	0	1	0	0	10
12	selama	TYPE	VOID	0	1	0	0	11
13	lakukan	TYPE	VOID	0	1	0	0	12
14	untuk	TYPE	VOID	0	1	0	0	13
15	ke	TYPE	VOID	0	1	0	0	14
16	turun_ke	TYPE	VOID	0	1	0	0	15
17	larik	TYPE	VOID	0	1	0	0	16
18	dari	TYPE	VOID	0	1	0	0	17
19	prosedur	TYPE	VOID	0	1	0	0	18
20	fungsi	TYPE	VOID	0	1	0	0	19
21	konstanta	TYPE	VOID	0	1	0	0	20
22	tipe	TYPE	VOID	0	1	0	0	21
23	kasus	TYPE	VOID	0	1	0	0	22
24	rekaman	TYPE	VOID	0	1	0	0	23
25	ulangi	TYPE	VOID	0	1	0	0	24
26	sampai	TYPE	VOID	0	1	0	0	25
27	writeln	PROCEDURE	VOID	0	1	0	0	26
28	readln	PROCEDURE	VOID	0	1	0	0	27
29	write	PROCEDURE	VOID	0	1	0	0	28
30	read	PROCEDURE	VOID	0	1	0	0	29
31	Ekspresi	PROGRAM	VOID	0	1	0	0	0
32	a	VARIABLE	INTEGER	0	1	1	0	0
33	b	VARIABLE	INTEGER	0	1	1	1	32
34	c	VARIABLE	INTEGER	0	1	1	2	33
35	hasil	VARIABLE	REAL	0	1	1	3	34
36	isTrue	VARIABLE	BOOLEAN	0	1	1	4	35

Input				
<pre>Block Table (btab): Idx Last Lpar Psize Vsize ----- 0 31 0 0 0 1 36 0 0 5 Array Table (atab): (empty)</pre>				
Penjelasan				
Analisis semantik berhasil memvalidasi ekspresi aritmatika dan logika yang melibatkan prioritas operator serta penggunaan tanda kurung. Decorated AST menunjukkan penanganan presedensi yang tepat serta resolusi tipe data yang akurat, di mana hasil perhitungan integer dapat diterima oleh variabel hasil yang bertipe real. Selain itu, ekspresi boolean majemuk yang melibatkan operator relasional dan logika sukses divalidasi tipe dan strukturnya, memastikan bahwa nilai akhir yang ditugaskan ke variabel isTrue konsisten dengan tipe boolean yang dideklarasikan di tabel simbol.				

3.3. Pengujian 3: Program dengan Syntax Error

Input	
<pre>program Error; variabel a, b : integer mulai a := 5 + ; jika a > 0 maka b := 1 selain_itu b := -1; writeln(b);</pre>	

Input
Output
<pre> --- Hasil Tokenisasi untuk error.pas --- Berhasil: Ditemukan 35 token. ----- ----- Memulai Syntax Analysis ----- [SYNTAX ERROR]: Berhenti. Pesan: Syntax Error: Expected SEMICOLON but got KEYWORD ('mulai') </pre>
Penjelasan
Parser berhasil mendeteksi pelanggaran sintaks. Meskipun tahap tokenisasi sukses mengidentifikasi 35 token, proses kompilasi dihentikan secara tepat saat kesalahan sintaks ditemukan, mencegah error ke tahap selanjutnya. Akibatnya, tahap Semantic Analysis tidak dieksekusi,

3.4. Pengujian 4: Program dengan Struktur Dasar

Input
<pre> program Hello; variabel a, b: integer; mulai a := 5; b := a + 10; writeln('Result = ', b); selesai. </pre>
Output

Input

```
===== DECORATED AST =====
ProgramNode(name: 'Hello')
├─ Declarations
│   ├── VarDecl('a') → tab_index:32, type:integer, lev:0
│   └── VarDecl('b') → tab_index:33, type:integer, lev:0
└─ Block → block_index:1, lev:1
    ├── Assign('a' := (5) → type:integer) → type:void
    │   ├── target 'a' → tab_index:32, type:integer
    │   └── value (5) → type:integer
    ├── Assign('b' := ('a'+10) → type:integer) → type:void
    │   ├── target 'b' → tab_index:33, type:integer
    │   └── value ('a'+10) → type:integer
    └─ ProcedureCall('writeln')
        ├── Char
        └── Variable
```

Input

Identifier Table (tab):

Idx	Name	Obj	Type	Ref	Nrm	Lev	Adr	Link

0	integer	TYPE	INTEGER	0	1	0	0	0
1	real	TYPE	REAL	0	1	0	0	0
2	boolean	TYPE	BOOLEAN	0	1	0	0	0
3	char	TYPE	CHAR	0	1	0	0	0
4	string	TYPE	STRING	0	1	0	0	0
5	program	TYPE	VOID	0	1	0	0	4
6	variabel	TYPE	VOID	0	1	0	0	5
7	mulai	TYPE	VOID	0	1	0	0	6
8	selesai	TYPE	VOID	0	1	0	0	7
9	jika	TYPE	VOID	0	1	0	0	8
10	maka	TYPE	VOID	0	1	0	0	9
11	selain_itu	TYPE	VOID	0	1	0	0	10
12	selama	TYPE	VOID	0	1	0	0	11
13	lakukan	TYPE	VOID	0	1	0	0	12
14	untuk	TYPE	VOID	0	1	0	0	13
15	ke	TYPE	VOID	0	1	0	0	14
16	turun_ke	TYPE	VOID	0	1	0	0	15
17	larik	TYPE	VOID	0	1	0	0	16
18	dari	TYPE	VOID	0	1	0	0	17
19	prosedur	TYPE	VOID	0	1	0	0	18
20	fungsi	TYPE	VOID	0	1	0	0	19
21	konstanta	TYPE	VOID	0	1	0	0	20
22	tipe	TYPE	VOID	0	1	0	0	21
23	kasus	TYPE	VOID	0	1	0	0	22
24	rekaman	TYPE	VOID	0	1	0	0	23
25	ulangi	TYPE	VOID	0	1	0	0	24
26	sampai	TYPE	VOID	0	1	0	0	25
27	writeln	PROCEDURE	VOID	0	1	0	0	26
28	readln	PROCEDURE	VOID	0	1	0	0	27
29	write	PROCEDURE	VOID	0	1	0	0	28
30	read	PROCEDURE	VOID	0	1	0	0	29
31	Hello	PROGRAM	VOID	0	1	0	0	0
32	a	VARIABLE	INTEGER	0	1	1	0	0
33	b	VARIABLE	INTEGER	0	1	1	1	32

Input

```
Block Table (btab):  
Idx  Last  Lpar  Psze  Vsze  
-----  
  0    31     0     0     0  
  1    33     0     0     2  
  
Array Table (atab):  
(empty)
```

Penjelasan

Analisis semantik berhasil memvalidasi alur program dasar yang mencakup deklarasi variabel, operasi assignment, dan pemanggilan prosedur standar. Decorated AST menunjukkan resolusi tipe yang tepat, di mana nilai literal dan hasil ekspresi aritmatika ($a + 10$) secara konsisten dipetakan ke tipe integer. Selain itu, pemanggilan `writeln` dengan argumen campuran (string dan variabel) berhasil divalidasi, dan Tabel Simbol mengonfirmasi alokasi memori yang benar untuk variabel lokal `a` dan `b` dengan offset alamat yang berurutan serta perhitungan ukuran blok yang akurat.

3.5. Pengujian 5 : Program dengan Loop dan Kondisional

Input
<pre>program Loop; Ferdin-Arsenic, variabel i, x : integer; mulai x := 0; untuk i := 1 ke 10 lakukan mulai x := x + i; jika (i = 5) maka writeln('Setengah jalan!') selain_itu writeln(i); selesai; selama x > 100 lakukan x := x - 1; selesai.</pre>
Output

Input

```
===== DECORATED AST =====
ProgramNode(name: 'Loop')
├── Declarations
│   ├── VarDecl('i') → tab_index:32, type:integer, lev:0
│   └── VarDecl('x') → tab_index:33, type:integer, lev:0
└── Block → block_index:1, lev:1
    ├── Assign('x' := (0) → type:integer) → type:void
    │   ├── target 'x' → tab_index:33, type:integer
    │   └── value (0) → type:integer
    ├── ForStatement (ke)
    │   ├── Number
    │   ├── Number
    │   └── CompoundStatement
    │       ├── Assign('x' := ('x'+'i') → type:integer) → type:void
    │       │   ├── target 'x' → tab_index:33, type:integer
    │       │   └── value ('x'+'i') → type:integer
    │       └── IfStatement
    │           └── RelationalExpression
    │               ├── Variable
    │               └── Number
    └── WhileStatement
        └── RelationalExpression
            ├── Variable
            └── Number
```

Input

Identifier Table (tab):

Idx	Name	Obj	Type	Ref	Nrm	Lev	Adr	Link

0	integer	TYPE	INTEGER	0	1	0	0	0
1	real	TYPE	REAL	0	1	0	0	0
2	boolean	TYPE	BOOLEAN	0	1	0	0	0
3	char	TYPE	CHAR	0	1	0	0	0
4	string	TYPE	STRING	0	1	0	0	0
5	program	TYPE	VOID	0	1	0	0	4
6	variabel	TYPE	VOID	0	1	0	0	5
7	mulai	TYPE	VOID	0	1	0	0	6
8	selesai	TYPE	VOID	0	1	0	0	7
9	jika	TYPE	VOID	0	1	0	0	8
10	maka	TYPE	VOID	0	1	0	0	9
11	selain_itu	TYPE	VOID	0	1	0	0	10
12	selama	TYPE	VOID	0	1	0	0	11
13	lakukan	TYPE	VOID	0	1	0	0	12
14	untuk	TYPE	VOID	0	1	0	0	13
15	ke	TYPE	VOID	0	1	0	0	14
16	turun_ke	TYPE	VOID	0	1	0	0	15
17	larik	TYPE	VOID	0	1	0	0	16
18	dari	TYPE	VOID	0	1	0	0	17
19	prosedur	TYPE	VOID	0	1	0	0	18
20	fungsi	TYPE	VOID	0	1	0	0	19
21	konstanta	TYPE	VOID	0	1	0	0	20
22	tipe	TYPE	VOID	0	1	0	0	21
23	kasus	TYPE	VOID	0	1	0	0	22
24	rekaman	TYPE	VOID	0	1	0	0	23
25	ulangi	TYPE	VOID	0	1	0	0	24
26	sampai	TYPE	VOID	0	1	0	0	25
27	writeln	PROCEDURE	VOID	0	1	0	0	26
28	readln	PROCEDURE	VOID	0	1	0	0	27
29	write	PROCEDURE	VOID	0	1	0	0	28
30	read	PROCEDURE	VOID	0	1	0	0	29
31	Loop	PROGRAM	VOID	0	1	0	0	0
32	i	VARIABLE	INTEGER	0	1	1	0	0
33	x	VARIABLE	INTEGER	0	1	1	1	32

Input				
<pre> Block Table (btab): Idx Last Lpar Psze Vsze ----- 0 31 0 0 0 1 33 0 0 2 Array Table (atab): (empty) </pre>				
Penjelasan				
Analisis semantik berhasil memvalidasi struktur kontrol alur yang kompleks, mencakup perulangan untuk dan selama serta percabangan jika yang bersarang di dalam blok majemuk. Decorated AST menunjukkan penanganan hierarki scope yang tepat, di mana variabel iterator <i>i</i> dan akumulator <i>x</i> terikat secara konsisten ke tipe integer pada tabel simbol di seluruh blok iterasi. Selain itu, ekspresi relasional pada kondisi loop dan percabangan sukses divalidasi tanpa konflik tipe, serta Tabel Blok (btab) mengonfirmasi kalkulasi ukuran memori variabel lokal yang akurat untuk mendukung eksekusi logika perulangan tersebut.				

4. Kesimpulan dan saran

4.1. Kesimpulan

Berdasarkan pelaksanaan Milestone 3 tugas besar mata kuliah IF2224 Formal Language and Automata Theory, implementasi *semantic analysis* untuk *compiler* Pascal-S telah berhasil dikembangkan. Implementasi ini memanfaatkan mekanisme penelusuran *Abstract Syntax Tree* (AST) dan pengelolaan Tabel Simbol (*Symbol Table*) untuk memvalidasi makna kontekstual dan logika program yang sebelumnya telah lolos verifikasi sintaksis.

Proses pengujian yang mencakup lima kasus uji unik (termasuk kasus dasar, alur kontrol, ekspresi kompleks, deklarasi, dan satu kasus error) menunjukkan bahwa *analyzer* dapat menegakkan aturan semantik bahasa dengan akurat. Modul ini terbukti mampu memverifikasi kompatibilitas tipe data dalam operasi aritmatika dan penugasan, mendeteksi penggunaan variabel yang belum dideklarasikan, serta mencegah terjadinya deklarasi ulang variabel yang tidak sah dalam lingkup yang sama.

Lebih lanjut, pengujian memvalidasi keselarasan antara argumen formal dan aktual pada pemanggilan prosedur atau fungsi, serta memastikan validitas tipe pada ekspresi kondisi dalam struktur kontrol. *Semantic analyzer* juga menunjukkan kemampuan *error handling* yang presisi dengan mendeteksi ketidakkonsistenan logika dan melaporkan *Semantic Error* secara informatif. Secara keseluruhan, implementasi ini berhasil memperkaya representasi struktur program menjadi *Decorated AST* dan *Symbol Table* yang valid dan siap digunakan untuk tahap selanjutnya, yaitu pembangkitan kode antara.

4.2. Saran

Berdasarkan evaluasi implementasi dan hasil pengujian, ada saran yang dapat diajukan untuk perbaikan dan pengembangan lebih lanjut:

1. Analisis Aliran Data Sederhana : Validasi saat ini berfokus pada kebenaran tipe dan lingkup. Pengembangan selanjutnya dapat mencakup analisis statis yang lebih mendalam, seperti mendeteksi variabel yang dideklarasikan tetapi tidak pernah digunakan (*unused variables*) atau mendeteksi kode yang tidak dapat dijangkau (*dead code*). Fitur ini akan memberikan peringatan yang berguna selain pesan kesalahan fatal.

5. Referensi

- [1] “Let’s Build A Simple Interpreter. Part 7: Abstract Syntax Trees”
<https://ruslanspivak.com/lbasi-part7> [Accessed: November 25th, 2025].
- [2] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson Education.
- [3] “Semantic Analysis (ULiège)”
<https://people.montefiore.uliege.be/geurts/Cours/compil/2015/04-semantic-2015-2016.pdf>
[Accessed: November 25th, 2025].
- [4] Institut Teknologi Bandung. Spesifikasi Tugas Besar IF2224 - Formal Language and Automata Theory, Milestone 3.  Spesifikasi Milestone 3 - Tubes IF2224 TBFO

Lampiran

- Link Release Repository Github.
<https://github.com/orvin14/JWA-Tubes-IF2224>
- Pembagian Tugas (menunjukkan persentase kontribusi).

Anggota	Pembagian Tugas
Muhammad Alfansya (13523005)	Testing, Laporan
Orvin Andika Ikhsan Abhista (13523017)	Testing, Laporan
Dzaky Aurelia Fawwaz (13523065)	Implementasi Semantic Analysis Testing, Laporan
Ferdin Arsenarendra Purtadi (13523117)	Implementasi Semantic Analysis, Testing, Laporan